MSSE 601 – SOFTWARE ENGINEERING FUNDAMENTALS
WEEK 4 Project Component and UI Design
Golden Bike Shop Project
Group Category: SE Fundamentals
Group Name: SEF 1 - NCM Innovators

Prepared by:
Minnette Johnson
Nasar Roble
Carolann Mora
Peter Olayiwola

Regis University
Submitted for: MSSE 601 – Software Engineering Fundamentals
Date: September 20, 2026

TABLE OF CONTENTS

1. Key User Journeys **2**
Journey 1 — Staff Look-up & Bike Detail Update 2
Journey 2 — Processing a New Rental 3
Journey 3 — Bike Check-in & Maintenance Dispatch 4
Why These Journeys Were Selected 4
Screen 1 — Inventory Management 5
Screen 2 — Bike Details / Edit 6
Screen 3 — New Rental 7
Screen 4 — Bike Return 8
3. Design Reasoning **9**
4. Accessibility **10**
Color 10
Keyboard Access 10
Text Size and Contrast 11
Error Messages 12
5. Team Decision Record **13**
Decision 1: Inventory Layout 13
Decision 2: Separate Bike Details/Edit Screen 13
Decision 3: Require a Status Decision After a Bike Return 13
Decision 4: Accessibility and Status Communication 14
6.1 Contributor AI Usage Summary **15**
AI Use Record 15
AI Use Record 16
7. Named Ownership **16**

1. Key User Journeys

Journey 1 — Staff Look-up & Bike Detail Update

User Persona & Goal:

Jake (Store Clerk) — Goal: Find an available e-bike for a walk-in customer at the Golden shop and review/update its details before initiating rental.

Steps Taken (User Actions):

Locate Bike: Jake accesses the Inventory Management table (Screen 1) and uses the search bar and filter controls to filter by Status: Available and Location: Golden.

Select & Inspect: Jake identifies a matching e-bike in the table and clicks the **"View / Edit Details"** action button.

Update Information: On the Bike Details / Edit screen (Screen 2), Jake checks the bike's battery level/condition and updates its notes before clicking **"Save Changes"**.

System Responses:

Filters and displays matching rows dynamically in the inventory table (Screen 1).

Opens Screen 2 populated with the selected bike's current database fields (Bike ID, Type, Status, Location, Condition).

Validates modified fields, saves updates to the database, and redirects Jake back to the active inventory view with a success notification.

Failure / Error Cases:

Inventory Collision: Another clerk reserves the bike simultaneously; system alerts Jake on Screen 1 that the bike status changed to Reserved and refreshes the table.

Form Validation Failure: Jake tries to save changes on Screen 2 with a blank location or invalid status; system blocks submission and highlights required fields.

Journey 2 — Processing a New Rental

Aligned Screen: Screen 3

User Persona & Goal: Jake (Store Clerk) — Goal: Record a new rental agreement for customer Marcus and dispatch the selected e-bike.

Steps Taken (User Actions):

Initiate Form: From the inventory table, Jake selects the available bike and clicks **"Rent Bike"**, opening the New Rental screen (Screen 3).

Enter Details: Jake inputs customer information (Marcus), selects rental start/expected return timestamps, and verifies payment status.

Confirm Checkout: Jake reviews the rental summary and clicks **"Confirm Rental"**.

System Responses:

Pre-populates Screen 3 with the selected Bike Name/ID and current timestamp.

Validates that all required customer fields and return dates are filled.

Updates the bike status in the database from Available to Rented, triggers the IoT lock release command payload to the bike, and generates a digital receipt.

Failure / Error Cases:

Missing Required Data: Jake leaves the customer name or return date blank; system prevents confirmation and displays an explicit error message (e.g., "Rental cannot be completed. Please enter the customer's name and return date").

Hardware Telemetry Failure: Smart wheel-lock fails to respond over cellular/Bluetooth; system alerts staff on Screen 3 to re-trigger the command or swap the bike assignment.

Journey 3 — Bike Check-in & Maintenance Dispatch

Aligned Screen: Screen 4

User Persona & Goal: Elena (Shop Technician) — Goal: Process a returned e-bike, record its mechanical/diagnostic condition, and decide its next status.

Steps Taken (User Actions):

Look up Active Rental: Elena opens the Bike Return screen (Screen 4) and scans/selects the returning bike.

Record Condition: Elena inspects the returned bike, reviews telemetry fault codes (e.g., motor error or battery wear), and selects the post-inspection condition (e.g., Maintenance Required).

Complete Return: Elena clicks **"Confirm Return"**.

System Responses:

Fetches active rental records, calculates final rental duration, and checks stationary lock telemetry.

Processes payment closing/overage fees and issues an automated receipt.

Updates bike status to Maintenance (removing it from available rental searches) or Available based on Elena's explicit choice.

Failure / Error Cases:

Late Return Overage: Return timestamp exceeds the reserved window; system calculates overage charge and prompts confirmation before final billing.

Unflagged Damage Safety Check: Elena attempts to mark the bike Available, but active motor fault codes exist in telemetry; system pops up a required confirmation modal forcing verification before restocking.

Why These Journeys Were Selected

This section provides explicit justification for selecting these specific core workflows over secondary operational paths:

- **Core Business Revenue & Fleet Flow:** Reservation, pickup/ride, and check-in cover 100% of the primary revenue-generating customer touchpoints and core fleet operations across all 4 store locations.
- **High-Risk Edge Case Validation:** These user paths directly exercise high-risk operational edge cases—such as inventory shortages during peak demand, telemetry/unlock command failures over cellular/Bluetooth, missing customer details, and unrecorded bike damage.

Safety & Customer Satisfaction: Ensuring real-time availability updates and preventative maintenance tracking directly prevents double-booking and avoids assigning faulty e-bikes, protecting store reputation and customer trust.

2. Interface Designs

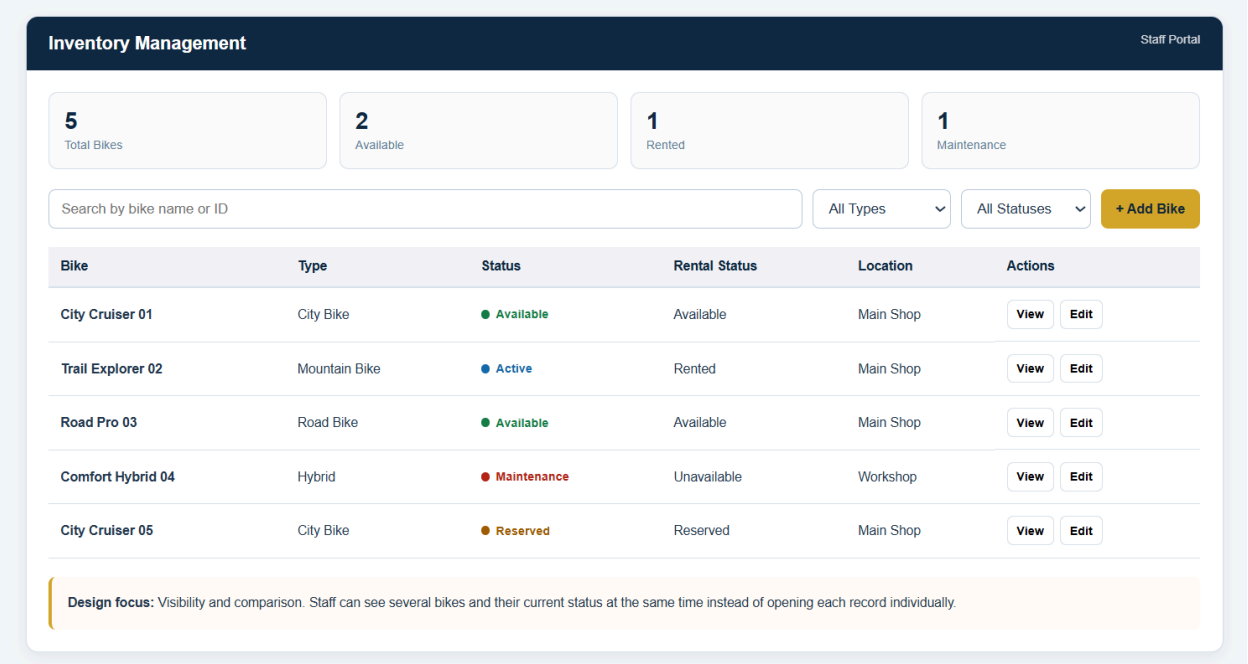
We designed four screens to support the three user journeys.

Screen 1 — Inventory Management

Supports: Journey 1
Requirement: Staff must be able to view, search, filter, and update bike inventory. Our original design uses a table because staff need to compare several bikes at once.

Main information shown:

- Bike
- Type
- Status
- Rental Status
- Location
- Actions
- Search
- Filters
- Add Bike

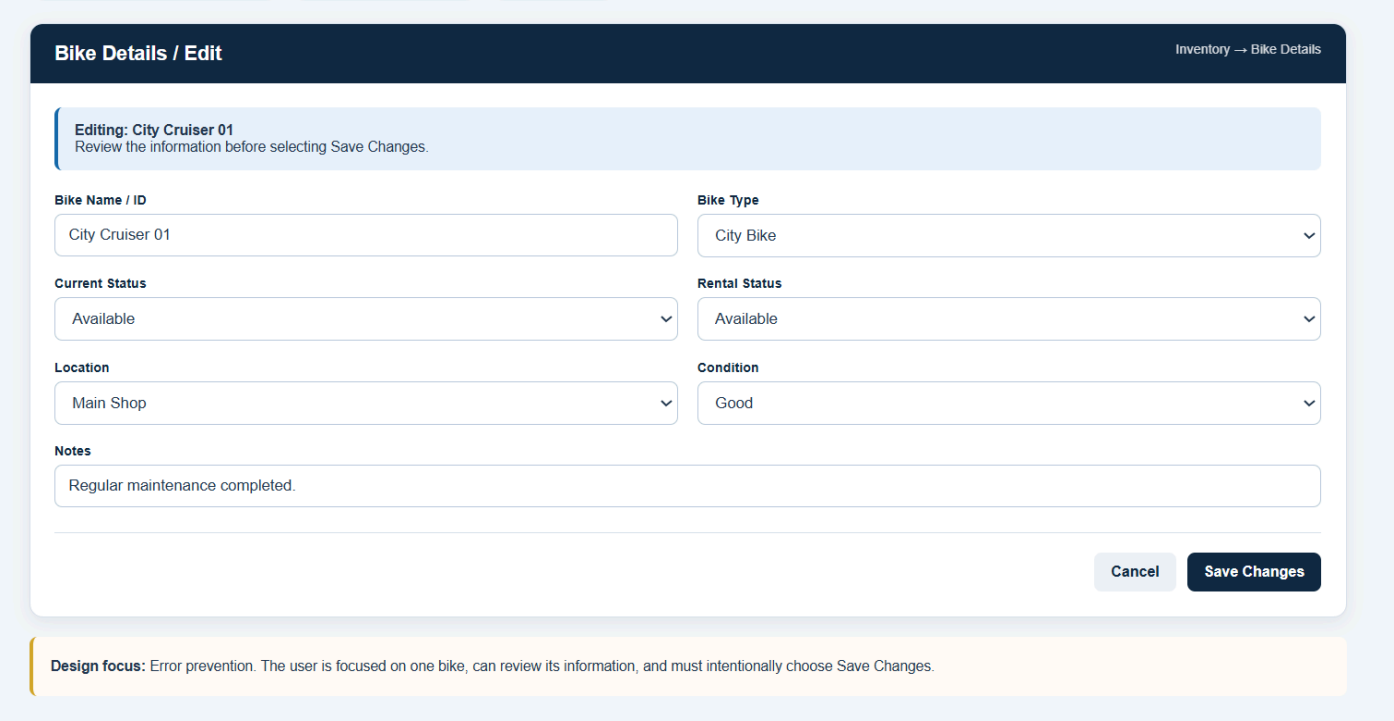


Screen 2 — Bike Details / Edit

Supports: Journey 1
Requirement: Staff must be able to view and update information about a bike. The screen allows staff to see the selected bike's information and make changes without having to search through the entire inventory again. The goal is to make updating one bike quick while keeping the user aware of which bike they are changing.

Main functions:

- Bike name/ID
- Bike type
- Current status
- Rental status
- Location
- Condition
- Save changes
- Cancel



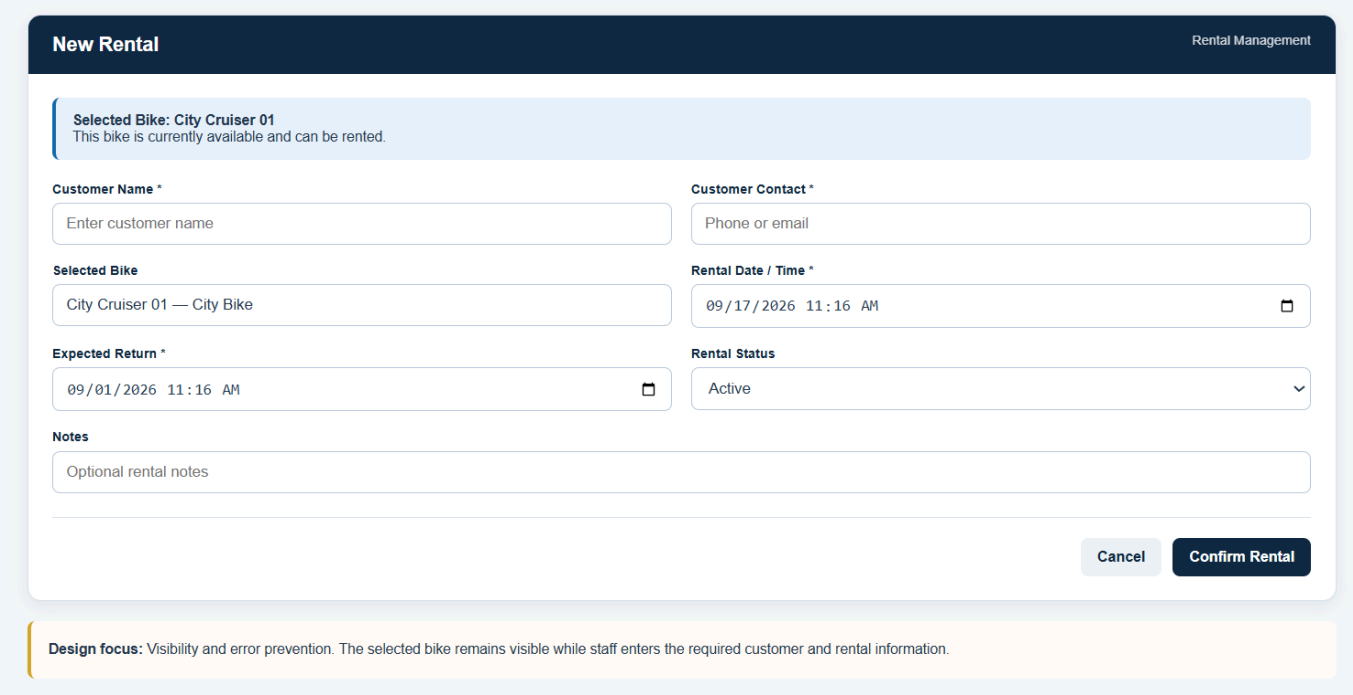
Screen 3 — New Rental

Supports: Journey 2
Requirement: Staff must be able to record a rental and update the selected bike's status.

The screen presents the selected bike and the information required to complete the rental. The system should prevent staff from completing the rental if required information is missing.

Main functions:

- Customer information
- Selected bike
- Rental date/time
- Expected return
- Rental status
- Confirm Rental
- Cancel



Screen 4 — Bike Return

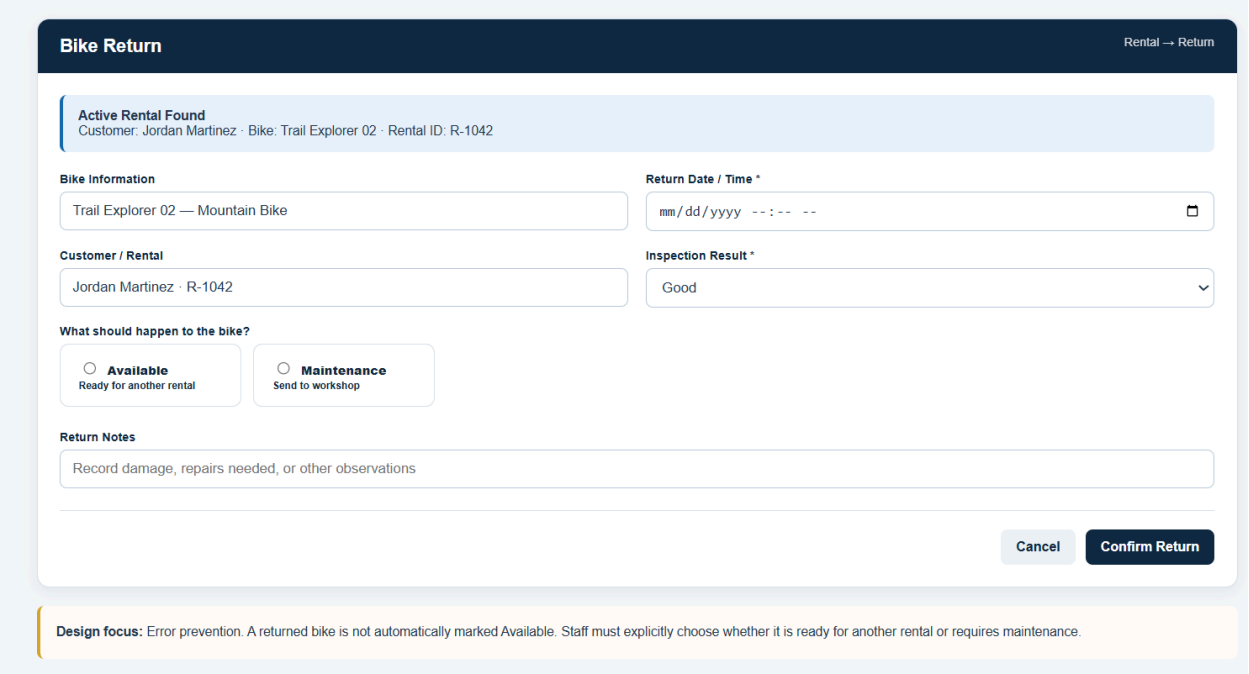
Supports: Journey 3
Requirement: Staff must be able to record a returned bike and update its condition/status.

The screen shows the active rental and allows staff to record the condition of the returned bike.

The important design decision is that the user must explicitly choose what happens to the bike after inspection rather than having the system automatically assume that every returned bike is immediately available.

Main functions:

- Rental information
- Bike information
- Return date/time
- Condition
- Available
- Maintenance
- Confirm Return



3. Design Reasoning

Screen	What We Were Trying to Make Easy	What We Accepted as Harder	Design Principle Applied	How We Applied It
Screen 1 – Inventory Management	We wanted staff to quickly see the status of several bikes and compare their type, rental status, and location without opening each bike individually.	A table gives less visual attention to an individual bike than a card-based layout, and users may need to scan across several columns.	**Visibility**	Bike status, rental status, location, and actions are displayed directly in the inventory table. Staff do not have to open each bike to find basic information.
Screen 1 – Inventory Management	We wanted staff to recognize the current status of a bike immediately instead of remembering or checking another screen.	The table contains several pieces of information, so it requires some scanning.	**Recognition rather than recall**	Statuses such as Available, Rented, Reserved, and Maintenance are displayed beside each bike using both text and visual indicators.
Screen 1 – Inventory Management	We wanted every bike to be presented in a predictable way so staff could quickly scan the inventory.	Using the same structure means some information may not be as prominent as it would be in a customized card.	**Consistency**	Every bike uses the same columns: Bike, Type, Status, Rental Status, Location, and Actions.
Screen 2 – Bike Details / Edit	We wanted staff to update one specific bike without accidentally changing another bike's information.	Staff have to open a separate screen instead of editing everything directly from the inventory table.	**Error prevention**	The selected bike is clearly identified, and changes are made through labelled fields before the user chooses **Save Changes**.
Screen 2 – Bike Details / Edit	We wanted staff to understand exactly what information they were changing.	The screen focuses on one bike, so staff cannot compare several bikes at the same time.	**Recognition rather than recall**	The bike name/ID, type, status, rental status, location, and condition are shown together.
Screen 3 – New Rental	We wanted staff to complete a rental without having to move between multiple screens to find the required information.	The form requires more information than the inventory screen, so completing a rental takes more steps.	**Logical grouping**	Customer information, selected bike information, rental dates, and rental status are grouped together on one screen.
Screen 3 – New Rental	We wanted staff to confirm that they are renting the correct bike before completing the transaction.	Keeping the selected bike visible takes up some additional screen space.	**Visibility**	The selected bike is displayed clearly at the top of the rental form while the staff member enters the rental information.
Screen 3 – New Rental	We wanted to prevent incomplete rentals from being submitted.	Staff must enter required information before they can complete the rental.	**Error prevention**	Required fields such as customer name, contact information, rental date, and expected return are clearly identified.
Screen 4 – Bike Return	We wanted staff to clearly decide what happens to a bike after it is returned.	The return process has an additional decision instead of automatically making every returned bike available.	**Error prevention**	Staff must choose whether the bike is **Available** or requires **Maintenance** before confirming the return.
Screen 4 – Bike Return	We wanted staff to see which customer and bike they were processing before recording the return.	The screen contains additional rental information that is not necessary for a simple status change.	**Visibility**	The active rental, customer, bike, and rental ID are displayed before the return is confirmed.
All Screens	We wanted users to understand the meaning of statuses without depending on colour vision.	Using both text and visual indicators takes slightly more space than using colour alone.	**Accessibility / Redundant cues**	Statuses use written labels such as “Available” and “Maintenance” alongside coloured indicators.
All Screens	We wanted users to understand what went wrong and how to fix it.	Detailed error messages take more space than a simple “Error” message.	**Clear feedback**	Error messages explain the problem and provide the next action, such as asking the user to enter missing rental information.

Overall, our design prioritizes visibility, consistency, recognition rather than recall, and error prevention. We focused on reducing the amount of information staff have to remember and the number of unnecessary steps needed to manage bikes, process rentals, and record returns. Where we

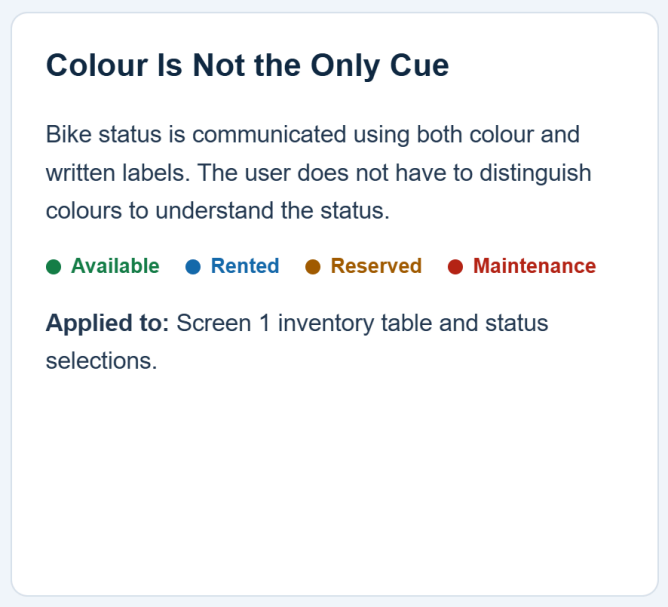
accepted additional steps, such as confirming a bike's condition during a return, we did so to reduce the possibility of incorrect inventory information. The alternatives we considered helped us identify these trade-offs and confirm which design choices were important to the Golden Bike Shop workflow.

4. Accessibility

Accessibility was considered across the four screens rather than treated as a separate feature. The practices below follow WCAG 2.1 AA guidance and are each tied to a specific screen from Section 2, so every claim can be checked directly against its screenshot.

Color

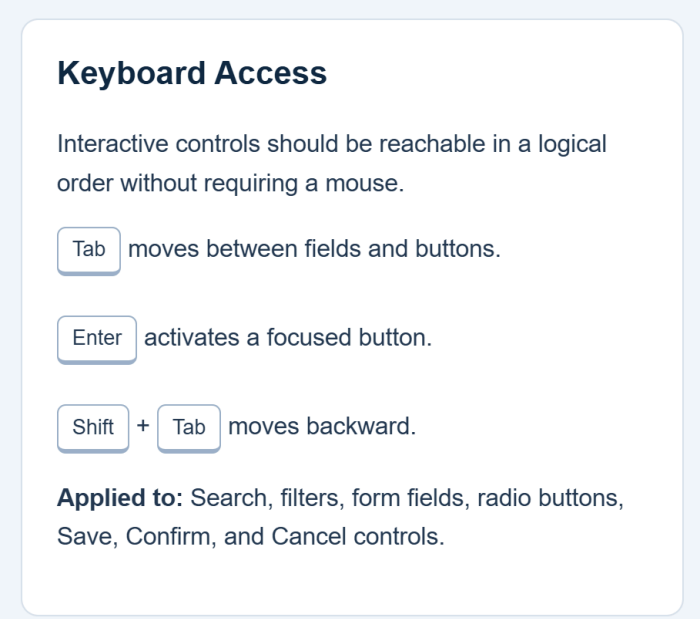
We do not rely on color alone to communicate bike status. Status labels such as **Available, Rented, Reserved, and Maintenance** are written as text alongside visual indicators. This pairing is used in Screen 1's inventory table, where every row shows a colored dot next to its status text, and in Screen 4's Available/Maintenance return choice, which uses labeled cards instead of color-only buttons. This means a user who cannot distinguish the colors can still understand the status.



Keyboard Access

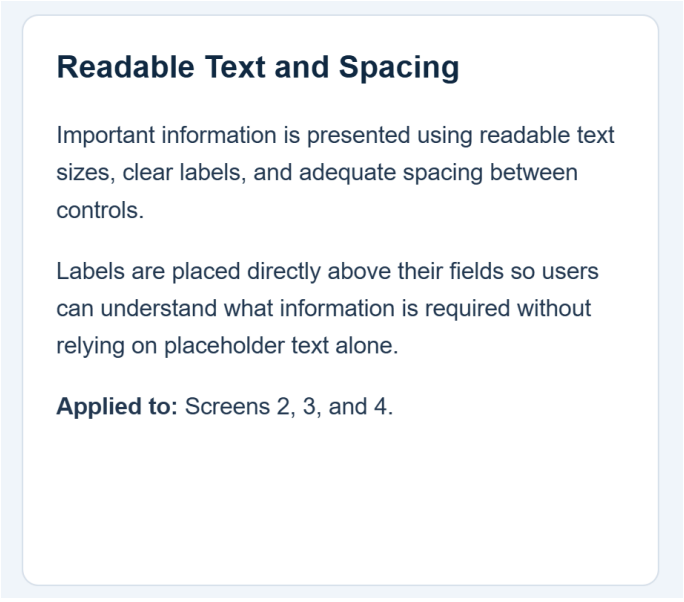
Buttons, search fields, filters, and form controls should be reachable using the keyboard. The interface should follow a logical tab order so staff can move through the screen without requiring a mouse. A visible focus indicator should mark the active control at each step, which matters most on Screen 4's Available/Maintenance selection: because those options use custom labelled cards rather

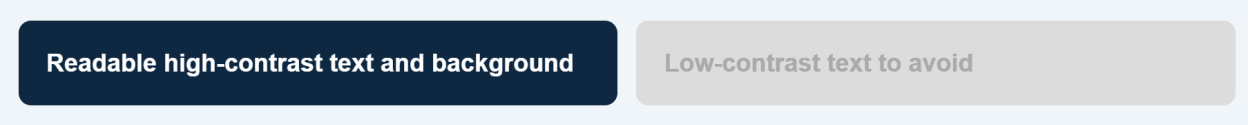
than native radio buttons, they need an explicit focus style so keyboard users can tell which option is currently selected.



Text Size and Contrast

We use readable text sizes and sufficient spacing between controls, with labels placed directly above their fields on Screens 2, 3, and 4 so staff are not relying on placeholder text alone. Text also needs enough contrast against its background to stay readable, not just large enough to read: the dark navy buttons, headers, and status text used across all four screens were chosen because they meet a stronger contrast ratio than light grey text on a light background, shown below.

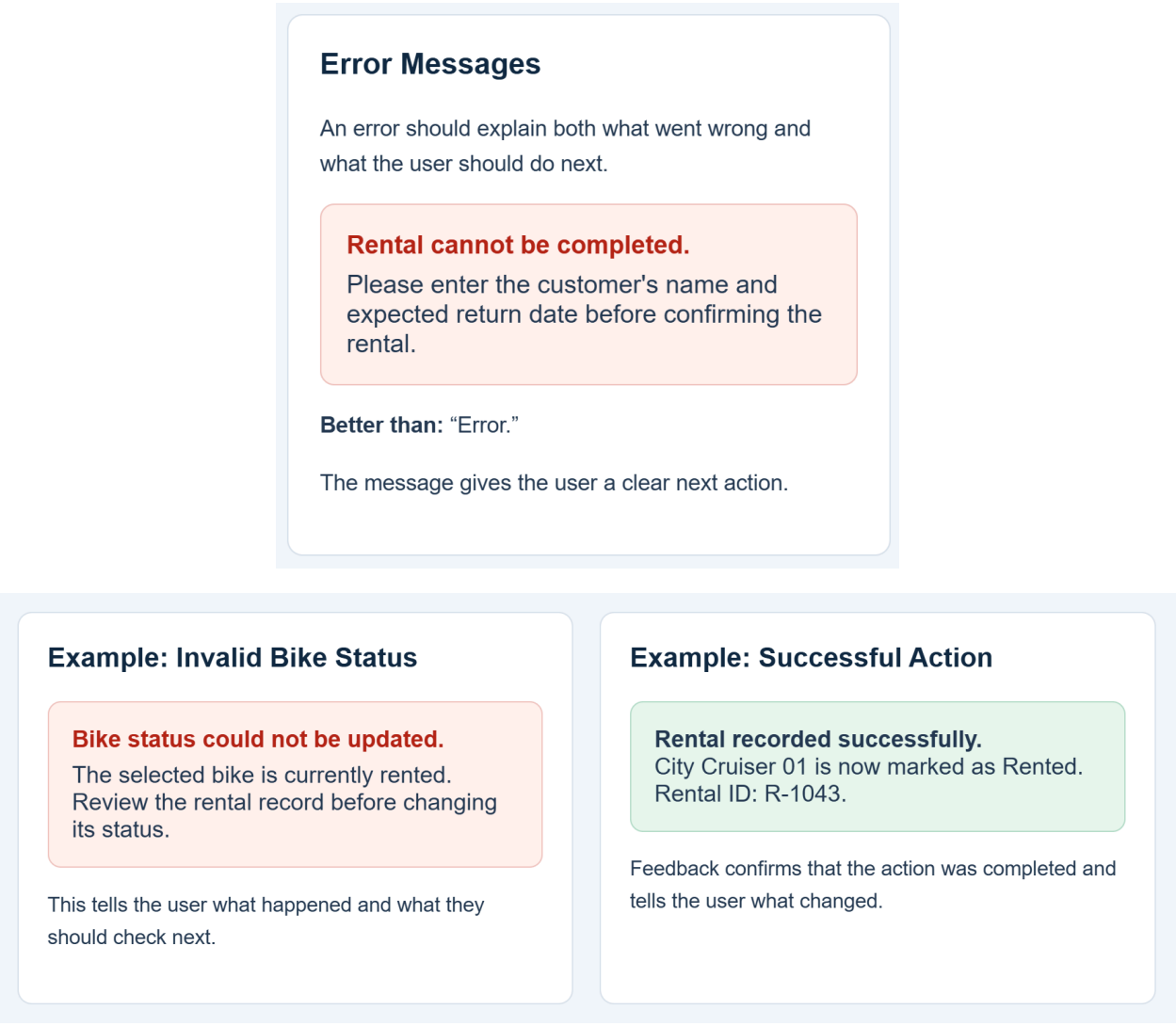




Error Messages

Errors should explain both **what went wrong and what the user should do next**.

For example, instead of "Error", the system should provide something like "Rental cannot be completed. Please enter the customer's name and return date." This gives the user a clear action to take.



These error-message examples directly support the Failure/Error Cases identified in Journey 2 (Screen 3) and Journey 3 (Screen 4), and reflect the same reasoning behind Decision 4 in the Team Decision Record (Section 5): status and error information should stay understandable through text, not rely on color or a single-word message alone.

5. Team Decision Record

Decision 1: Inventory Layout

The team considered using either a card-based view where bikes would be viewed individually or a table that displays multiple bikes at the same time. We chose a table-based inventory screen that shows the bike, type, status, rental status, location, and available actions, along with search and filter controls.

We chose the table because staff need to quickly compare several bikes and see their current status without opening each bike individually. This reduces the amount of information the user has to remember and supports Adobe's principle of reducing cognitive load through recognition rather than recall. The trade-off is that the screen contains more information at once and can appear denser than a card-based design. This decision did not require a major change to the Week 3 requirements because it mainly changed how the existing inventory requirement was presented.

Decision 2: Separate Bike Details/Edit Screen

The team considered allowing staff to edit bike information directly from the inventory table or opening a separate screen for editing. We chose a separate Bike Details/Edit screen. This screen clearly identifies the selected bike and provides labeled fields for reviewing and changing its information before the user selects Save Changes.

We chose this approach because it reduces the risk of staff accidentally changing the wrong bike and gives the user a chance to review the information before saving. This supports the good design principle that interfaces should be predictable and should help prevent errors before they occur. The trade-off is that staff have to take an additional step to open the edit screen instead of editing directly from the inventory table. This did not require a major Week 3 requirement change because the original requirement already called for staff to be able to view and update bike information.

Decision 3: Require a Status Decision After a Bike Return

The team considered automatically marking every returned bike as Available or requiring staff to choose whether the bike should be Available or placed in Maintenance. We chose to require staff to explicitly select the bike's status after inspecting it. The return screen was designed so that a returned bike is not automatically assumed to be ready for another rental.

We chose this option because a returned bike may be damaged or require maintenance. Automatically making every returned bike available could result in incorrect inventory information. Requiring an explicit decision keeps the user in control and helps prevent errors, both of which are emphasized in Adobe's UI design guidance. The trade-off is that the return process takes slightly longer because staff must make an additional decision. This design work refined the Week 3

requirement so that the system should not automatically mark a returned bike as available; staff must determine its status after inspection.

Decision 4: Accessibility and Status Communication

The team considered using color alone to represent bike status or using color together with written labels. We chose to use both written status labels and visual indicators. For example, statuses such as Available, Rented, Reserved, and Maintenance are displayed as text instead of depending only on different colors.

We chose this approach so that users who have difficulty distinguishing colors can still understand the bike's status. Adobe specifically recommends that color should not be the only way information is communicated and that other cues should be provided as well. The trade-off is that written labels require slightly more screen space than color-only indicators. This design work refined the Week 3 non-functional requirements by adding that important status information should not depend on color alone and should remain understandable through text or another visual cue.

Decision	Options and Choice	Reason and Trade-off	Week 3 Requirement Impact
1. Inventory Layout	Considered individual bike cards or a table. We chose a table-based inventory screen with bike information, status, search, filters, and actions.	A table lets staff compare several bikes at once and reduces the need to remember information between screens. The trade-off is a denser screen with more information displayed at once.	**No major change**. The existing inventory requirement stayed the same; only the way the information is presented changed.
2. Bike Details/Edit Screen	Considered editing directly in the inventory table or using a separate screen. We chose a separate Bike Details/Edit screen.	A separate screen makes it clear which bike is being edited and reduces accidental changes. The trade-off is an extra navigation step before editing.	**No major change.** The design supports the existing requirement to view and update bike information.

3. Bike Return Status	Considered automatically marking returned bikes as Available or requiring staff to choose Available or Maintenance. We chose an explicit status selection.	A returned bike may need repairs, so requiring a decision helps prevent incorrect inventory information. The trade-off is one additional step during the return process.	**Requirement refined**. Staff must choose the bike's status after inspection instead of the system automatically marking it Available.
4. Accessibility and Status	Considered using color alone or combining color with written labels. We chose color plus text labels such as Available, Rented, Reserved, and Maintenance.	Text labels make status understandable for users who cannot distinguish colors. The trade-off is slightly more screen space.	**Non-functional requirement refined.** Important status information should not depend on color alone.

6. Contributor AI Usage Summary

- **Contributor:** Nasar Roble
- **What we asked AI:** We asked ChatGPT to suggest alternatives to our existing UI designs and help organize them into a Team Decision Record using Week 4 requirements and Adobe UI principles.
- **What AI gave us:** AI suggested alternative design options, explained their benefits and trade-offs, and created a draft Team Decision Record.
- **What we changed and why:** We kept four important decisions that directly related to our screens. We removed less important items, shortened the explanations, and adjusted the wording to match our project. We also removed suggestions that were not part of our design and out of scope. The final decisions were made by the team.
- **Owner:** Nasar Roble owned the AI-assisted Team Decision Record and AI Use Record.
- **Contributor: Peter Olayiwola**

AI Use Record

Entry 2: User Journey Edge-Case Alignment (Section 1)

- **Technique Used:** GAP ANALYSIS & CRITIQUE
- **Tool & Version:** Gemini 1.5 Pro
- **What We Asked For:**
  "Review our three draft staff user journeys (Look-up & Edit, New Rental, and

Return/Check-in) against our four interface screen designs (Inventory, Details, New Rental, Bike Return). Point out any logical disconnects, missing validation errors, or edge cases between user steps and system responses."

- **What It Returned:** The tool pointed out a disconnect in Journey 2 where the checkout process did not explicitly state how hardware unlock failures or missing customer return dates were communicated back to the user.
- **What We Changed and Why:** We updated Journey 2 and Journey 3 to include explicit **Failure / Error Cases**. Specifically, we added explicit system error messages (e.g., "Rental cannot be completed. Please enter the customer's name and return date") and added a confirmation modal when a technician attempts to return a bike to Available status while active telemetry fault codes exist.
- **Owner:** Peter Olayiwola (Key User Journeys Lead)
- **Contributor: Carolann Mora**

AI Use Record

Entry 3: Section 4 Accessibility Alignment (Section 4)

- **Technique Used:** GAP ANALYSIS & CROSS-REFERENCING
- **Tool & Version:** Claude (Anthropic), Sonnet 5
- **What We Asked For:**
  "Review our Section 4 Accessibility draft against screenshots and point out anywhere it does not align."
- **What It Returned:** The tool found that Section 4 described accessibility practices in general terms without tying them to specific screens they applied to, that the “Text Size” heading was paired with the screenshot that was actually about color contrast rather than text size, and that the status vocabulary used in the section (Available, Rented, Reserved, Maintenance) does not match Screen 1’s Status column, which displays “Active” for the rented bike.
- **What We Changed and Why:** We added explicit references to Screen 1 and Screen 4 in the Color and Keyboard Access subsections, renamed “Text Size” to “Text Size and Contrast” and rewrote it to cover both screenshots, and added a closing paragraph linking the Error Messages examples to the Failure/Error Cases in Journeys 2 and 3 and to Decision 4 in the Team Decision Record.
- **Owner:** Carolann Mora (Accessibility Lead)

Contributor: Minnette Johnson

- **Contributor:** Minnette Johnson

Entry 4: Interface Design Alternatives and Design Reasoning (Sections 2 and 3)

- **Technique Used:** DESIGN ALTERNATIVE COMPARISON & CRITIQUE

- **Tool & Version:** ChatGPT (GPT-5.6)
- **What We Asked For:**
  "Review our proposed Golden Bike Shop inventory interface and compare the table-based design with a card-based alternative. Consider the Week 4 requirements, including staff needing to search, filter, compare multiple bikes, view status and location, and access bike actions. Identify the strengths, weaknesses, and trade-offs of each approach and explain which design considerations should guide the final interface."
- **What It Returned:** The tool identified that a table-based inventory layout supports staff who need to compare multiple bikes quickly because key information such as bike name, type, status, rental status, location, and actions can be viewed together. It also identified the card-based design as an alternative that could provide greater visibility for an individual bike but would require staff to review more individual items when comparing several bikes. The analysis also highlighted the trade-off between information density and individual-item visibility and suggested that search, filtering, and clear visual hierarchy could reduce the cognitive load created by a dense table.
- **What We Changed and Why:** We used the analysis to evaluate the alternatives against the actual Golden Bike Shop workflow rather than selecting a design based only on appearance. We retained the table-based inventory design because the primary task requires staff to compare several bikes at once and quickly identify status, location, and available actions. We incorporated search and filter controls to make the larger amount of information easier to navigate. We also used the comparison to strengthen the Design Reasoning section by explaining the trade-off between information density and individual-bike visibility instead of simply stating that the table was preferred.
- **Who Owned the Change:** Minnette Johnson (Interface Design and Design Reasoning Lead)

7. Named Ownership (Peter)

Individual Accountability & Deliverable Ownership: While every team member is responsible for drafting and owning their assigned individual section (Section 1 and 7 by Peter Olayiwola, Section 2 and 3 by Minnette Johnson, Section 5 and 6 by Nasar Roble), as the designated Week 4 Deliverable Owner, it is **Peter Olayiwola's** responsibility to ensure overall document consistency, synthesize contributions, and actively resolve non-uniform ideas across sections. This includes ensuring that the user journeys written in Section 1 directly map to the interface designs in Section 2 and that technical rationale aligns across the Decision Record.

Rotation Model Alignment: Per NCM Innovators' Team Charter rotation model, deliverable leadership rotates weekly to ensure all team members lead across the term:

- **Week 1 (Team Charter):** Minnette Johnson
- **Week 2 (Project Charter & Process Selection):** Nasar Roble
- **Week 3 (Requirements Specification):** Carolann Mora

Week 4 (Interface Design): Peter Olayiwola (First-time deliverable lead)